SVARA: An Integrated Platform for Karaoke Processing, Vocal Assessment, and Guided Singing Practice

S. Sneha
Roll No.: 241701055
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, Tamil Nadu, India

B. Mahati
Roll No.: 241701029
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, Tamil Nadu, India

Under the Guidance of Dr. P. Revathi
Department of Computer Science and Design
Rajalakshmi Engineering College, Chennai, Tamil Nadu, India

***Abstract*—Singing practice commonly spans several disconnected activities: obtaining an accompaniment track, recording a learner, estimating pitch, following lyrics, and completing structured exercises. SVARA—Karaoke and Learn—is an integrated mobile platform designed to connect these activities through karaoke playback, vocal analysis, and guided learning. Its Flutter/Dart client communicates with a Python FastAPI backend and Supabase-related services. The audio workflow incorporates Demucs and UVR/MDX-Net-family source-separation components, including the model file `UVR_MDXNET_KARA_2.onnx`. Reference melody pitch is estimated with librosa and pYIN, while learner pitch is estimated with a custom YIN-based method. The reported implementation configuration includes 44.1-kHz input, 4096-sample buffers, RMS-based voice-activity gating, a 40-ms RMS window, an approximately 5.5% peak threshold, a voiced-confidence threshold near 0.60, and a pitch tolerance of approximately 55 cents with a three-frame stability condition. Timestamped transcription uses faster-whisper with `large-v3`. The learning concept also incorporates Indian classical music topics such as swaras, ragas, the 22-Sruti framework, and varisai exercises, while not treating every concept as an automated feature. Preliminary formative testing involved approximately 20–30 people from varied backgrounds. No controlled accuracy or learning-gain results are available; therefore, this paper documents system integration and its current engineering parameters rather than claiming validated pedagogical effectiveness. Known limitations include possible transposition inconsistency, a gamaka parsing issue, incomplete section detection, in-memory XP persistence, and security validation work.**

***Index Terms*—singing assessment, fundamental frequency, YIN, pYIN, music source separation, karaoke, lyric transcription, computer-assisted music learning, Indian classical music**

I. Introduction

Digital music tools have made accompaniment playback and audio recording widely accessible, but the practice loop for a learner can remain fragmented. A user may need one application to isolate a song's vocals, another to follow lyrics, and a separate tool or teacher to understand pitch deviation. SVARA is designed to connect these activities in one workflow: users can practise with karaoke audio, inspect pitch-related feedback, and access guided exercises within a single mobile application.

The contribution is primarily one of *system integration*, not a claim of a new pitch estimator or source-separation algorithm. SVARA combines established audio-analysis methods with a practice-oriented interface and an intended bridge between contemporary songs and Indian classical learning concepts. Its three connected modules are vocal analysis, karaoke, and learning. Vocal analysis estimates pitch and compares a learner contour with a reference; karaoke supports source-separated playback and practice controls; learning organizes basic vocal exercises and concepts such as swaras and varisai.

A. Problem statement and motivation

A useful practice environment must coordinate audio input, accompaniment, pitch estimation, lyric timing, and learner-facing feedback. Each component has different constraints. Source separation can leave residual interference; pitch trackers can fail on breathy, noisy, or ornamented singing; transcription timestamps do not guarantee exact alignment to sung phonemes; and a single equal-tempered pitch reference does not fully describe all nuances of Indian classical performance. SVARA addresses the integration problem while retaining these limitations as explicit engineering concerns.

II. Related Work

YIN estimates fundamental frequency using a modified difference-function approach intended to reduce common octave and period errors [1]. pYIN extends YIN by representing multiple pitch candidates probabilistically and using sequence decoding to improve temporal consistency [2]. These methods provide established foundations for frame-wise pitch tracking, but their outputs still depend on signal quality, voicing, and the musical context.

Music source separation has progressed from signal-processing approaches to neural systems. Demucs uses a hybrid

waveform/spectrogram architecture for music demixing [3]. UVR provides a user-facing ecosystem of separation models, including MDX-Net-family models; model choice and inference path affect the separation result. Such models support karaoke preparation, but separation quality is not equivalent to perfect isolation.

For speech transcription, Whisper was trained using large-scale weak supervision and supports multilingual transcription [4]. SVARA uses faster-whisper with the `large-v3` model for timestamped transcription. Transcription is useful for lyric display, but speech recognition should not be assumed to produce exact sung-lyric or melody alignment, especially under accompaniment, melisma, and ornamentation. The librosa library provides implementations of audio analysis functions, including pYIN and feature extraction [5].

Computer-assisted music learning is a broad area spanning feedback interfaces, automated performance analysis, and interactive practice. SVARA's current scope is a practical prototype integrating pitch feedback with karaoke and structured exercises. A controlled comparison with teacher ratings or annotated reference performances remains future evaluation work; the present prototype data do not establish effectiveness.

III. Proposed System and Architecture

Fig. 1 summarizes the logical system. The Flutter client handles user interaction, audio capture, playback controls, and presentation of results. API requests connect it to a Python/FastAPI backend. Supabase is used for backend-related data and service integration. Python processing components handle separation, reference-pitch analysis, learner-pitch analysis, and timestamped transcription. The architecture is modular: karaoke processing and pitch feedback share the song/practice context, but their models and outputs remain distinct.

A. Connected practice workflow

Fig. 2 shows the core analysis loop. A song/reference and a learner recording enter separate estimation paths. The resulting pitch tracks are compared in a common time frame where possible, and the application displays a feedback state. This comparison requires careful handling of unvoiced frames and any transposition applied to the accompaniment or reference.

IV. Methodology

A. Audio preprocessing and source separation

The karaoke workflow uses neural source separation to obtain vocal and accompaniment components from a mixed song. Demucs and UVR/MDX-Net-family processing are part of the project workflow, and the provided model filename `UVR_MDXNET_KARA_2.onnx` identifies one model artifact. The project description does not establish that all models are run concurrently; therefore, the separation path is described as model-dependent orchestration rather than a single fixed ensemble.

Let a stereo or mono mixture be represented by $x(t)$. A separation model estimates source components $\hat{s}_k(t)$, where k may denote vocals or accompaniment stems. In general,

$$x(t) \approx \sum_{k=1}^{K} \hat{s}_k(t) + e(t), \tag{1}$$

where $e(t)$ represents residual error and artifacts. This is a conceptual mixture relation, not a claim that the estimated stems reconstruct the input exactly. The separated accompaniment supports karaoke playback, while the vocal stem can support reference or inspection workflows where appropriate.

B. Reference pitch estimation with pYIN

The reference path uses librosa and pYIN with a configured search range of approximately 65–1000 Hz and a hop length of 1024 samples. For each frame, pYIN estimates a fundamental frequency and a voicing-related confidence measure. Its probabilistic candidate handling and sequence decoding are useful for obtaining a temporally coherent contour [2], though reference pitch can still be affected by polyphony, accompaniment leakage, vibrato, and separation artifacts.

For a periodic signal with sample rate f_s, a pitch period τ corresponds approximately to

$$f_0 = \frac{f_s}{\tau}. \tag{2}$$

In practice, τ is estimated from a signal representation and the pitch tracker selects among candidate periods or frequencies. The reported pYIN range limits the frequencies considered by the reference analysis; it should not be interpreted as a validated vocal range for every user.

C. Learner pitch estimation and voice activity gating

The learner path uses a custom YIN-based approach at a 44.1-kHz sampling rate with 4096-sample audio buffers. RMS-based voice activity gating uses a 40-ms RMS window and an approximate peak threshold of 5.5%. A voiced-confidence threshold near 0.60 is also configured. These are implementation parameters, not universal thresholds or experimentally optimized values.

For a frame $x[n]$ of length N, RMS energy is

$$E_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} x[n]^2}. \tag{3}$$

The gate helps avoid issuing pitch feedback during silence or low-energy input. The YIN difference function for lag τ can be written as

$$d_t(\tau) = \sum_{j=0}^{W-1} \left(x_t[j] - x_t[j+\tau]\right)^2, \tag{4}$$

where W is the comparison window and x_t is the current frame. A low difference at a plausible lag suggests periodicity. The original YIN method applies additional normalization and candidate-selection steps [1]; Eq. (4) illustrates its core principle rather than reproducing every implementation detail.

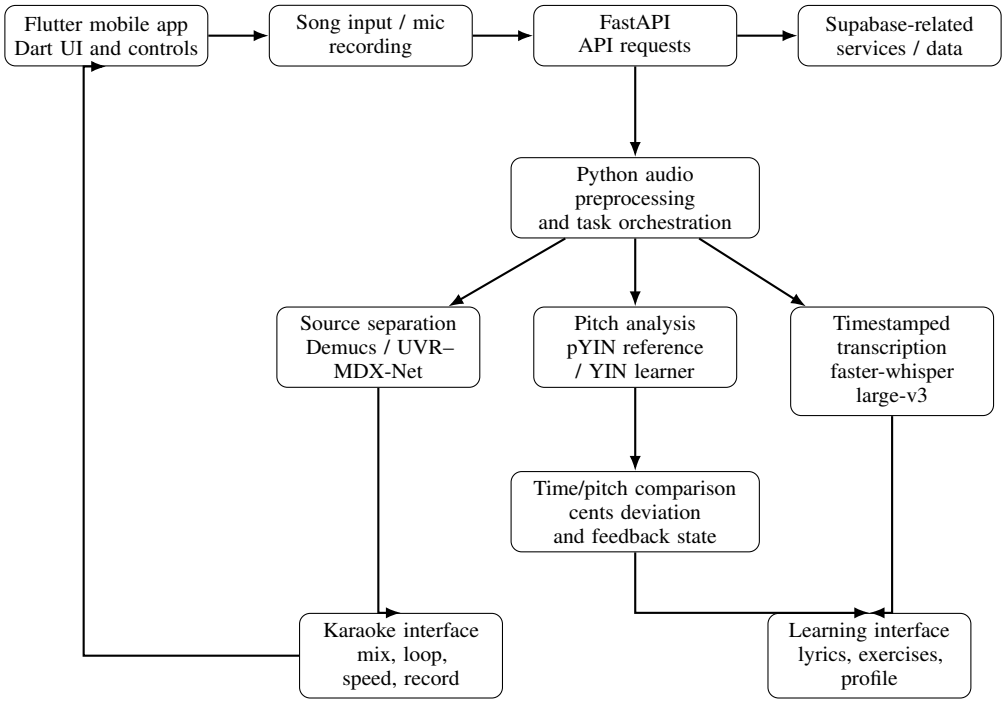


Fig. 1. Logical architecture of SVARA. The diagram shows component responsibilities and data flow; it does not imply that every separation model executes simultaneously or that every learning concept is automated.

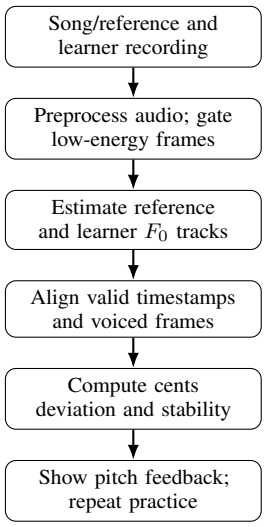


Fig. 2. Audio-to-feedback workflow. Invalid or unvoiced frames should be excluded rather than interpreted as low-pitch singing.

D. Pitch representation and feedback

Frequency can be mapped to an equal-tempered MIDI note number using

$$m = 69 + 12 \log_2 \left(\frac{f}{440} \right), \tag{5}$$

where f is frequency in hertz and MIDI note 69 corresponds to 440 Hz. For reference frequency f_r and learner frequency f_u, signed deviation in cents is

$$C = 1200 \log_2 \left(\frac{f_u}{f_r} \right). \tag{6}$$

Positive values indicate that the learner estimate is higher than the reference; negative values indicate lower pitch. A tolerance of approximately 55 cents is configured for feedback. The system also uses three consecutive frames as a stability condition and configures two attempts per note. These values are implementation choices; no evidence supplied here establishes them as optimal.

The implementation description reports MIDI scanning beginning around MIDI 48/60 and extending to 96, with downward scanning to 36. Because this range is described as configuration behavior rather than a fully specified algorithm, its precise search ordering and interaction with transposition should be validated in testing. Time-aligned comparison can be expressed as

$$C_i = 1200 \log_2 \left(\frac{f_u(t_i)}{f_r(t_i)} \right), \tag{7}$$

for timestamps t_i where both tracks contain valid voiced estimates. When the tracks are not aligned or either estimate is unvoiced, the comparison should be withheld rather than assigned an artificial error.

V. Karaoke Playback and Lyrics

The karaoke interface includes playback speed options of 0.5×, 0.75×, and 1×, a four-channel mixer, looping, recording, and background-music controls. A playback drift

TABLE I
SELECTED IMPLEMENTATION PARAMETERS

Component	Reported configuration
Reference pitch	librosa pYIN; approx. 65–1000 Hz; hop length 1024 samples
Learner pitch	Custom YIN-based method; 44.1 kHz; 4096-sample buffers
Voice activity	RMS window 40 ms; peak threshold approx. 5.5%
Voicing decision	Confidence threshold approx. 0.60
Pitch feedback	Tolerance approx. 55 cents; 3 consecutive frames
Practice attempts	2 attempts per note configured
MIDI scanning	Around MIDI 48/60 to 96; downward scan to 36
Playback	0.5×, 0.75×, 1×; 4-channel mixer
Drift checking	Approx. every 3 s; threshold approx. 180 ms
Lyrics	faster-whisper `large-v3`, timestamped transcription
Separation	Demucs and UVR/MDX-Net family; `UVR_MDXNET_KARA_2.onnx`

TABLE II
FEATURE STATUS BASED ON THE AVAILABLE PROJECT DESCRIPTION

Status	Items and qualification
Implemented / reported	Flutter interface; FastAPI API layer; Supabase integration; source-separation workflow; reference pYIN and learner YIN-based pitch paths; pitch feedback configuration; playback controls; timestamped transcription workflow.
Partial / requires validation	Transposition consistency between client and analysis endpoint; gamaka parsing; section detection; persistence of XP and progress; synchronization behavior under varied loads.
Future / not established as complete	Validated 22-Sruti recognition; robust automated raga/gamaka interpretation; reliable section detection; systematic lyric-to-melody alignment; measured accuracy guarantees.

check is performed approximately every three seconds with a reported threshold of about 180 ms. This check is intended to detect noticeable divergence between playback components; it does not establish a measured synchronization accuracy. The four-channel mixer provides a user-facing mechanism for balancing available audio channels or stems according to the implementation's routing.

Timestamped transcription uses faster-whisper with the `large-v3` model. Transcription segments can be represented as $(w_i, t_i^{start}, t_i^{end})$, where w_i is the recognized text and the two times delimit a segment. These timestamps can support lyric-following interfaces and practice prompts. They do not, by themselves, guarantee word-level alignment to singing or exact correspondence between syllables and melody notes. Multilingual performance depends on language, recording conditions, accompaniment, and singing style.

VI. Indian Classical Music-Based Learning

SVARA's learning direction includes foundational Indian classical music concepts alongside song-based practice. Swaras provide named pitch functions within a musical framework; ragas organize characteristic melodic material and movement; and structured exercises such as Sarali Varisai, Janta Varisai, and Daattu Varisai can support progressive practice of pitch patterns and transitions. Lip trills and breathing exercises are included as basic vocal-practice activities.

The 22-Sruti framework is an important theoretical account of microtonal distinctions in Indian music. It should not be collapsed into a claim that every Sruti is implemented as a fixed frequency or that the application automatically identifies all 22 Srutis. A 12-tone equal-temperament scale divides an octave into equal logarithmic steps, whereas the 22-Sruti concept reflects a distinct theoretical and performance tradition whose interpretation varies by context. Gamaka refers to ornamented pitch movement, not merely a static note label. Robust computational representation of gamaka requires contour and context beyond nearest-note matching. In SVARA, these concepts inform the learning design; the current implementation review identifies a gamaka parsing issue, so automated gamaka support should not be presented as complete.

VII. Prototype Implementation

The mobile client is built with Flutter and Dart. The Python backend uses FastAPI for API development and Supabase for backend-related data and service integration. Audio-processing components are implemented in Python and combine source-separation models with pitch-estimation and transcription libraries. This separation of concerns allows the application UI to coordinate tasks without embedding all analysis logic in the mobile interface.

VIII. Preliminary User Feedback and Evaluation

The prototype has been tried by approximately 20–30 people from varied backgrounds, including students, teachers, music teachers, amateur users, singers, and one sound engineer. This is treated as preliminary formative evaluation: diversity of perspectives can reveal workflow friction, terminology problems, playback usability concerns, and differences between casual and musically experienced users. The available information does not include exact category counts, a standardized questionnaire, quantitative ratings, controlled conditions, or recorded performance metrics. Accordingly, no percentages, satisfaction scores, accuracy figures, or learning gains are reported.

A future evaluation should specify recruitment and inclusion criteria, use a fixed set of reference melodies, compare estimated pitch against manually verified annotations, and report measures such as voiced-frame coverage, gross pitch error, median absolute cents deviation, and transcription timing error. A separate learning study could compare repeated practice outcomes with a suitable baseline and include teacher assessment. Any such study should predefine tasks, record device and acoustic conditions, and report uncertainty rather than relying only on subjective impressions.

IX. Discussion and Limitations

The system's main value proposition is the connection of accompaniment preparation, pitch feedback, timestamped lyrics, and structured exercises. This may reduce context switching during practice, but the prototype has not yet been shown to

improve singing performance quantitatively. Source separation may introduce vocal leakage or artifacts that affect pitch analysis. Pitch trackers can confuse harmonics, miss low-energy notes, and respond poorly to breathiness, vibrato, or rapid ornamentation. The pYIN reference path and custom YIN learner path also differ methodologically, so their outputs should not be assumed to have identical error characteristics.

The technical review identifies a possible transposition mismatch between the client and analysis endpoint. If the displayed accompaniment is transposed but the comparison reference is not transformed consistently, feedback may systematically indicate an offset. A gamaka parsing issue limits confidence in ornament-specific handling. Section detection may remain a stub rather than a complete capability. XP persistence is in-memory rather than fully persistent, and bearer-token handling and job-ownership checks require further security validation. These are open limitations, not solved features.

X. Future Work

Priority work includes (1) resolving transposition consistency across client and backend, (2) evaluating pitch tracking across vocal ranges and recording conditions, (3) improving handling of gamaka and ornamented singing, (4) validating lyric-to-melody alignment, (5) completing song-section detection, (6) persisting learning progress and history, and (7) strengthening bearer-token validation and job ownership. A larger controlled study should evaluate pitch estimates against annotated references and assess learning outcomes over repeated sessions. The learning interface can also be refined with music-teacher input so that feedback remains understandable without reducing nuanced musical performance to a single pitch score.

XI. Conclusion

SVARA—Karaoke and Learn—is an integrated prototype connecting karaoke playback, audio source separation, vocal pitch assessment, timestamped transcription, and guided singing practice. Its reported implementation combines a Flutter/Dart client, FastAPI backend, Supabase-related services, Demucs and UVR/MDX-Net-family separation, librosa pYIN reference estimation, custom YIN-based learner estimation, and faster-whisper transcription. Indian classical concepts provide a learning direction, but the paper distinguishes conceptual scope from completed automation. Preliminary testing across approximately 20–30 diverse users provides formative feedback only; without controlled measurements, no accuracy or learning-effectiveness claim is warranted. Resolving known integration, persistence, and security issues and conducting systematic evaluation are necessary next steps.

References

[1] A. de Cheveigné and H. Kawahara, "YIN, a fundamental frequency estimator for speech and music," *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002, doi: 10.1121/1.1458024.

[2] M. Mauch and S. Dixon, "pYIN: A fundamental frequency estimator using probabilistic threshold distributions," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 659–663, doi: 10.1109/ICASSP.2014.6853678.

[3] A. Défossez, "Hybrid spectrogram and waveform source separation," arXiv:2111.03600, 2021.

[4] A. Radford *et al.*, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Machine Learning*, Proc. Machine Learning Research, vol. 202, 2023, pp. 28492–28518.

[5] Librosa Development Team, "librosa: Audio and music signal analysis in Python." [Online]. Available: https://librosa.org/doc/

[6] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2." [Online]. Available: https://github.com/SYSTRAN/faster-whisper

[7] Meta Research, "Demucs: Music source separation." [Online]. Available: https://github.com/facebookresearch/demucs